Part B — Production Architecture

My plan for turning the little Amazon-checkout prototype into something that could actually run as a product — for 100+ customers, across apps that change every sprint, still trustworthy on the 50th run six months from now.

Before any of the detail, here's the one belief the whole thing rests on. The crawler is a guesser — it only finds what it happens to bump into. The graph is the opposite — it's structured, so it can reason about what *ought* to be there. I don't want production trusting either of them blindly. So browser agents do the sensing and clicking, plain deterministic code does the validating and storing, and the graph is the thing that decides what's still risky, what's missing, and what's quietly gone stale.

One thing I want to be upfront about: this isn't architecture astronomy. I'm not drawing boxes I've never built. Many of the core claims already have working seeds in Part A, and I've put a table at the end (section 11) pointing at the exact function that backs each one — the table maps the production design back to the working prototype.

What the prototype already does	What it has to become in production
browser-use + a DFS frontier	An agent harness that picks what to test, constrains the action, checks the outcome, and keeps replayable traces.
Neo4j concepts / intents / scenarios	A temporal, living graph: provenance, confidence, absence modeling, PR blast-radius traversal.
Graph-inferred missed scenarios	A deterministic rule engine with eval-calibrated confidence and a promotion workflow.
Amazon checkout only	One feature modeled deeply; the same pattern repeated per customer/app/feature under hard tenant isolation.

0 · The short version

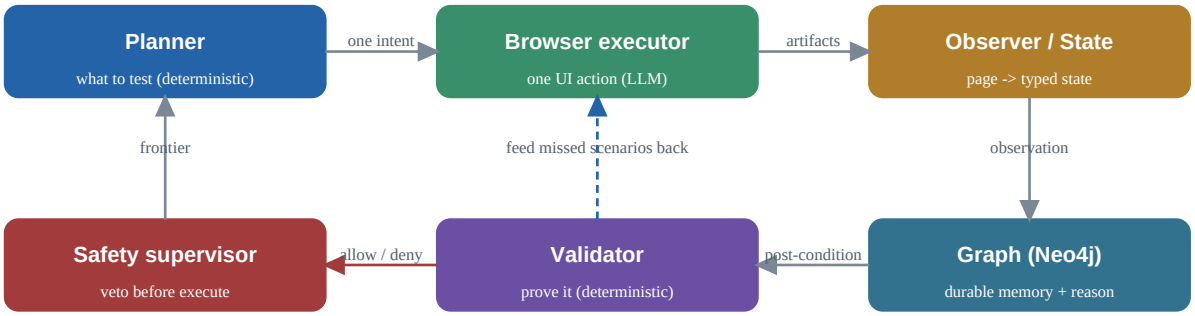
Let me start with what the prototype already does, because the production plan is really just "preserve this loop, but harden it for production conditions." The browser agent walks through Amazon checkout, everything it sees gets written into Neo4j as typed facts, and then the graph points out behaviours the agent skipped — like "does changing the quantity update the subtotal?" or "does deleting an item?" And it doesn't just list them in a report; it hands the doable ones back to the agent, which goes and tests them. In a real run it genuinely does this for the quantity change.

Production keeps that same division of labour and toughens it up. The browser agent stays a guesser that we never treat as the source of truth. The graph stays the long-term memory, and it's not allowed to make anything up beyond what was actually observed plus the rules we've vetted. In between sits a deterministic harness that owns the parts you can't leave to a model: what to test, what's safe, what state we're in, what to re-run, and what counts as proven.

Here's the production failure mode I worry about most. It isn't the agent missing a button — you can always crawl again. It's **confident wrongness**: the agent swears checkout worked when it really hit a login wall; a button's selector quietly changed after a deploy and nobody noticed; the graph quietly assumes a feature was tested when it never even appeared. Every design choice in here is bent toward dragging that kind of quiet uncertainty into the open where you can see it.

Decision	Planning, safety, validation, ingestion, replay and graph reasoning are deterministic services. The LLM / browser-use is used only for perception and one narrow UI action at a time.
Trade-off	I give up some apparent autonomy. In return I get auditability and repeatability — and I still absorb DOM churn, because the executor leans on the visible page, ARIA and screenshot rather than brittle selectors.
Refusal line	I would not ship a system where browser-use's own "done" is enough to mark a scenario validated.

Figure 1 — the loop: agents act, deterministic services decide what becomes truth.



Three things break this loop once you leave the demo, and they organise the rest of the document: the app changes (so the schema needs versioning, §2, and the graph needs a freshness policy, §3); both layers are probabilistic (so you need an eval harness, §4); and cost compounds per customer per crawl (so models get routed to tasks under hard budgets, §5).

1 · Agent decomposition and boundaries

The biggest call I made is to split responsibilities instead of building one clever agent. A single agent that plans, clicks, reasons, validates, writes to the graph and then narrates itself looks wonderful in a demo and comes apart the first week you operate it. So this is a harness of small services, each with a contract it cannot step outside of.

Service	Owns	LLM?	The boundary I enforce
Planner	Frontier, priority, budgets, expected states, replay requests	No	Decides what to test; it never clicks.
Browser executor	One UI action via browser-use (DOM/ARIA/screenshot/context)	Yes	Runs one intent, returns artifacts; never writes graph truth.
Observer	Page snapshot → state, actions, forms, raw artifacts	Mostly no	A small classifier is fine, but it must hand back the raw evidence.
State resolver	Authoritative state from URL + DOM + ARIA + graph + summary	No	The browser-use opinion is an input, not the verdict.
Validator	Deterministic post-conditions after an action	No	Required before "executed" is allowed to become "validated".
Reasoner	Missed / absent scenarios from graph structure	No online	Online path is Cypher; an LLM only proposes new rules offline.
Safety supervisor	Allow-list, forbidden final-payment, destructive controls	No	Vetoes before browser-use ever sees the task.
Ingestor	Idempotent graph writes, provenance, confidence, artifact refs	No	No model calls; every write is typed and replayable.

The boundary I care about most: the executor is allowed to *say* "I clicked Proceed to Checkout." The validator still has to *prove* we reached a checkout boundary, a sign-in wall, or a failure, from independent evidence. The executor's words are evidence, not a verdict. That is precisely what the prototype already does in `_checkout_reached_ok` and `_verify_cart_mutation` — it re-observes and asserts instead of believing the model's summary.

I also keep safety as its own gate, separate from planning, so a planning bug can never quietly remove a safety check. The canonical risk on an intent wins; an LLM suggestion can only make it *more* restrictive, never less. In the prototype that is `normalize_suggestion` ignoring any risk the model tries to attach — a model cannot talk a destructive action into looking "safe".

One more boundary, and it is the one that decides whether this is a platform or just an Amazon script: app-specific quirks must never leak into the core. The prototype's smart-wagon cart URLs, marketplace cart differences and subtotal parsing all belong in a per-feature/app adapter. The core owns only the invariant machinery — the action contract, safety, evidence, graph writes, confidence and eval. The honest production test is whether the same core can drive Amazon checkout and, say, a SaaS settings-save flow with only adapter changes; if a new app forces a core change, the boundary is in the wrong place.

1.1 · The runtime contract

Every item the planner hands the executor carries a tight contract. This was the single change that turned the prototype from "the agent wandered off" into a disciplined executor.

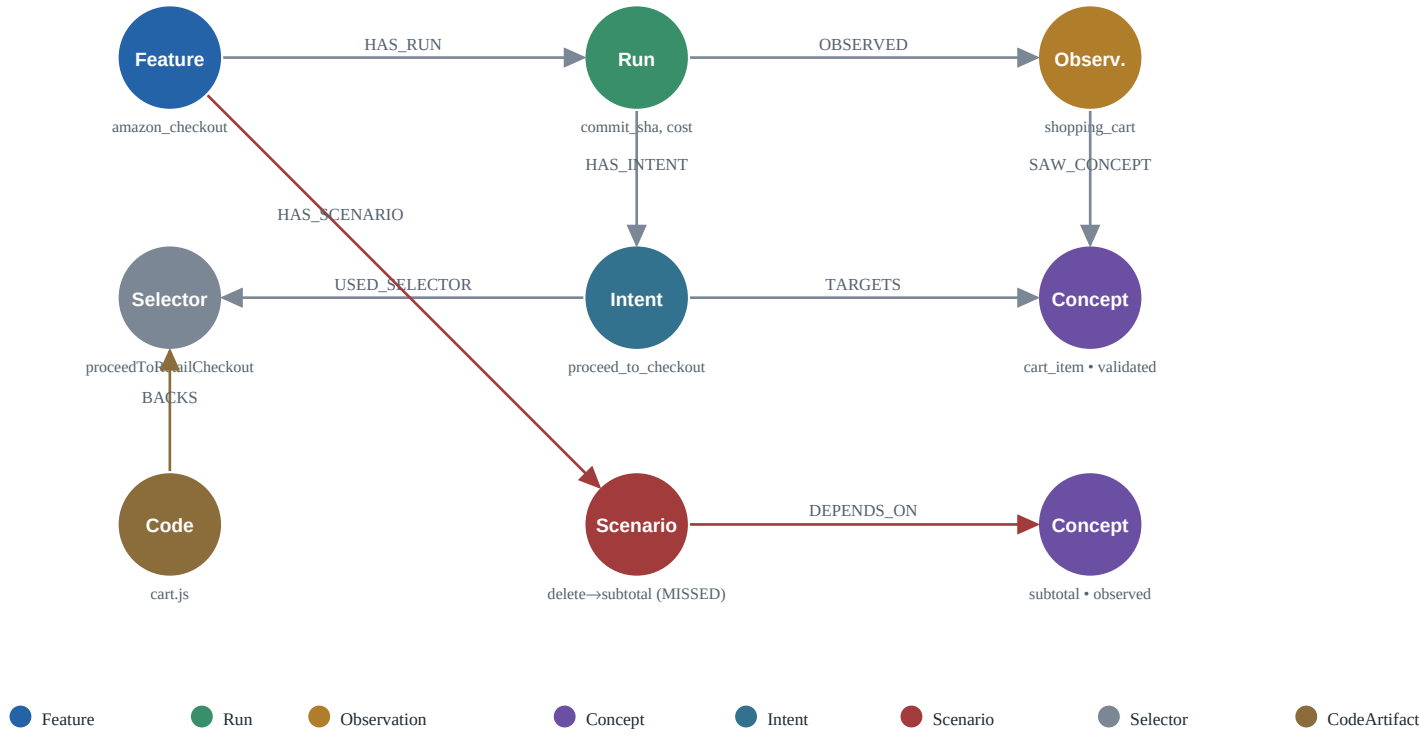
Field	Example	Why it exists
intent_id	PROCEED_TO_CHECKOUT	No dotted internal keys browser-use can mistake for a URL.
expected_state	shopping_cart	Stops it hunting for cart controls on a product or checkout page.
positive target	"Proceed to checkout / retail checkout"	Gives the executor a semantic target to find.
negative targets	Add to Cart, logo, search, Buy Now, Place Order	Heads off the usual wrong-clicks.
success evidence	checkout URL, secure checkout, address/payment	Keeps "clicked" and "validated" as separate ideas.

Field	Example	Why it exists
risk	safe / mutating / destructive / forbidden	Drives the safety and replay policy.
budget	max one UI-changing action	Stops the executor from re-planning mid-task.

2 · The production graph schema

The obvious thing to reach for here is the textbook hierarchy — Element → Component → Flow → Feature. I deliberately didn't use it as the backbone. It's decent vocabulary, but it's the wrong shape for the questions this platform actually has to answer: what did we observe, what did we genuinely prove, what should be there but isn't, what changed, and which tests depend on which pieces. So I built the graph around those questions instead. And the single most important choice is that the unit of meaning is a *behaviour* — a Concept like "add-to-cart" — not a DOM element. Elements are the first thing to break every time a site gets redesigned; if I anchored the graph to them it would rot constantly. Anchoring it to what a control *means* keeps it stable across redesigns.

Figure 2 — a slice of the production graph as Neo4j would draw it (Amazon checkout).
CONFIRMED_BY and RESOLVES_TO exist too (see §2.2); omitted here so the slice stays readable.



2.1 · Node types

Node	Key properties	Query it answers
Tenant / App / Feature	tenant_id, app_id, feature_key	Everything is tenant/app scoped; one feature modeled deeply.
Run	run_id, commit_sha, started_at, model/prompt version, cost	What was known at run N vs N+1; canary and rollback analysis.
Observation	state, url, artifact_ref, screenshot_ref, step	Replay and audit exactly what the browser saw.
Concept	key, kind, expected, observed, executed, validated, confidence, first_seen, last_seen	Expected-not-observed; observed-not-validated; stale concepts.
Intent	source, status, risk, selector_ref, evidence, confidence	Why an action ran, failed, or was blocked.
Scenario	feature-scoped key, status, kind, confidence, last_confirmed_run	Which scenarios are trusted right now versus decayed.
Selector	hash, css/xpath/role, quality, stability, last_failed	PR blast radius and self-healing.
CodeArtifact	path, symbol, commit_sha	Map a PR change to selectors, concepts and scenarios.

2.2 · Edges

```
(:Tenant)-[:OWNS]->(:App)-[:HAS_FEATURE]->(:Feature)
(:Feature)-[:HAS_RUN]->(:Run)-[:OBSERVED]->(:Observation)-[:ON_STATE]->(:PageState)
(:Observation)-[:SAW_CONCEPT]->(:Concept)
(:Run)-[:HAS_INTENT]->(:Intent)-[:TARGETS]->(:Concept)
(:Intent)-[:USED_SELECTOR]->(:Selector)-[:RESOLVES_TO]->(:Concept)
(:Feature)-[:HAS_SCENARIO]->(:Scenario)-[:DEPENDS_ON]->(:Concept)
(:Scenario)-[:CONFIRMED_BY]->(:Run)
(:CodeArtifact)-[:BACKS]->(:Selector)
```

2.3 · Three changes from the prototype, each earning its keep

1. **Scenario becomes feature-scoped, not run-scoped,** with CONFIRMED_BY edges back to runs. That is what lets the graph answer "is this scenario still true, and which commits confirmed it?" The prototype's run-scoped scenarios can only say "a run once produced this."
2. **Selector becomes a first-class node** (the prototype keeps it as a property on the intent). Now the PR blast radius is a plain traversal, CodeArtifact → Selector → Concept → Scenario. The prototype's `pr_blast_radius.py` is the contract-only seed of exactly this.
3. **Time and commit_sha live on everything.** That turns "show me concepts that existed at commit A but were never observed after commit B" into a graph diff — regression detection for free.

2.4 · Indexes and constraints I would actually declare

```
CREATE CONSTRAINT concept_key FOR (c:Concept) REQUIRE
  (c.tenant_id,c.app_id,c.feature_key,c.key) IS NODE KEY;
CREATE CONSTRAINT run_key FOR (r:Run) REQUIRE
  (r.tenant_id,r.app_id,r.feature_key,r.run_id) IS NODE KEY;
CREATE CONSTRAINT selector_key FOR (s:Selector) REQUIRE (s.tenant_id,s.app_id,s.hash) IS NODE KEY;
CREATE RANGE INDEX run_time FOR (r:Run) ON (r.started_at);
CREATE RANGE INDEX concept_seen FOR (c:Concept) ON (c.last_seen);
```

The DOM and screenshot blobs do not belong in Neo4j — Observation.artifact_ref points at per-tenant object storage. The graph stays a reasoning index, not a document store. (The prototype truncates text into the node to keep things simple; production moves it out.)

2.5 · Modeling absence — the part most designs skip

Absence is not the same as missing data. For a testing platform it is often the product itself: "we expected a promo-code capability and never saw it", or "delete was here last week and vanished after a PR". So expected concepts get materialized before they are ever observed (the prototype already does this in `seed_expected_concepts`), and absence comes in three flavours, each a one-line query.

Type	Definition	Example
Structural	expected ∧ ¬observed	The expected checkout boundary was never reached.
Behavioural	observed ∧ ¬validated	The quantity control exists, but the subtotal update was never proven.
Regression	last_seen < latest run	The promo field used to exist; gone after commit X.

The prototype already ships the first two (`missing_expected_concepts` and the inference query). Regression absence is the one that catches a sprint quietly deleting a field.

2.6 · The reasoning query (this is the real one, not a sketch)

Rules are data, not code — a set of prerequisites plus the pivot action whose validation would prove the behaviour. A scenario is "missed" when every prerequisite was observed but the pivot was never validated:

```
UNWIND $rules AS rule
OPTIONAL MATCH (c:Concept {tenant_id:$t, app_id:$a, feature_key:$f})
  WHERE c.key IN rule.requires AND coalesce(c.observed,false)=true
WITH rule, collect(DISTINCT c.key) AS present
WHERE size(present) = size(rule.requires)  // all prerequisites observed
OPTIONAL MATCH (pv:Concept {tenant_id:$t, app_id:$a, feature_key:$f, key:rule.pivot})
WITH rule, pv
```

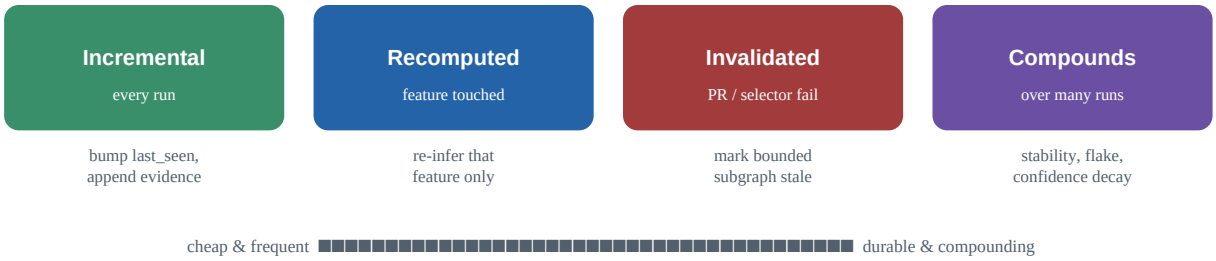
```
WHERE rule.pivot IS NULL OR pv IS NULL OR coalesce(pv.validated,false)=false  // pivot unproven
RETURN rule.key, rule.title, rule.status, rule.requires
```

Because the online reasoning is deterministic Cypher, it cannot hallucinate. It can only be wrong about a *rule* — which is exactly why §4 evaluates the rules, not individual inferences.

3 · Keeping the graph honest over time

This is the part the brief really pushes on, and rightly so: how is the graph still correct on the 50th run, six months and 30 deploys later? My answer is to stop pretending facts are permanent. A behaviour we confirmed six months ago is simply worth less than one we confirmed after last night's deploy. So "living" isn't a vibe — it's a concrete policy about what happens to each piece of data: some of it just gets topped up as we go, some gets recomputed, some gets thrown out and re-checked, and some quietly compounds into the thing that's actually valuable.

Figure 3 — the living-graph policy: four jobs with different costs and triggers.



Operation	Trigger	Implementation
Incremental	every crawl run	Upsert observations, bump last_seen, append evidence, update concept rollups, add Scenario-[:CONFIRMED_BY]->Run. O(seen this run).
Recomputed	a run touching a feature	Re-run inference and scenario confidence for that feature only. Never global.
Invalidated	PR touches mapped code, selector fails N runs, repeated validation failure	Mark a bounded subgraph stale; push the affected scenarios into the retest frontier.
Compounds	many runs over time	Selector stability, flake rate, transition probabilities, confidence-decay curves — the six-month moat.

3.1 · Confidence decay and retest

Every scenario's confidence decays unless something renews it: roughly `conf = w1·recency + w2·confirm_count − w3·flake_rate − w4·stale_dependencies`. A scenario nobody has re-confirmed in K runs slips below threshold and is automatically re-queued for the crawler — which closes the loop the prototype's "living graph" section only reports on. Nothing is true forever; truth has a half-life, and the system pays to renew it.

Decision	Invalidate bounded subgraphs; never nightly-recompute the whole customer graph.
Trade-off	It needs careful provenance and dependency edges to do well, but the cost and debuggability are far better — and invalidation actually *detects* drift instead of papering over it.
Refusal line	I would refuse to ship nightly full-graph recomputation as the main freshness mechanism.

4 · How I'd know any of it is actually right

There are two places this system can be confidently wrong, and they need different checks. The browser agent can miss things or fumble a click. And the graph rules can infer something that looks structurally sensible but is actually useless. So I check both, separately — and the golden rule is that "the agent said it worked" is never where the story ends.

Figure 4 — trust ladder: nothing is "validated" on the executor's word alone.

Question	Signal	Metric
Did it click the right target?	browser-use trace + target + negative-target check	wrong-target rate, wander rate
Did the action succeed?	deterministic post-condition after re-observation	validation pass rate
Is the scenario reproducible?	replay across clean sessions + golden apps	flake rate, pass@k
Are confidence scores calibrated?	predicted confidence vs observed reproduction	ECE, Brier score
Did a model/prompt change regress?	canary evals on versioned golden trajectories	delta vs baseline

I would run three eval layers. Offline **golden apps** with human-labeled states, actions and scenarios — precision and recall of discovered scenarios is the agent's regression metric. **Canary crawls** before any model or prompt change ships. And **production sampling**, where a small slice of high-impact actions is checked by deterministic assertions plus the occasional human spot-check. One nuance worth stating: because inference is deterministic, I evaluate the *rules*, not each inference — every rule carries a precision score on golden data, and an LLM-proposed rule sits in shadow mode until it earns its way in.

Decision	A confidence number shown to a customer must be calibrated.
Trade-off	Calibration costs you a feedback loop and some humility about early numbers.
Refusal line	If scenarios we label 0.90 only reproduce 65% of the time, that number does not go in the product UI.

5 · What this costs, and which model does what

The strategy here is emphatically not "use the smartest model everywhere" — that is how inference cost becomes uncontrolled. It's matching each job to the cheapest thing that can actually do it, with real budgets. Most of the work is plain code and costs nothing. The high-volume clicking goes to a small, fast model. The big expensive model only comes out for the rare, genuinely hard stuff — discovering new rules offline, settling eval disputes — where its reasoning earns its price.

Task	Default	Why
State resolver, validation, inference, ingestion, frontier	**Deterministic (free)**	~90% of operations; must be explainable and repeatable. Never use a model to check a model.
Browser execution (which element to click)	**Small/fast — GPT-4o-mini / Haiku-class / Gemini Flash**	High volume, low stakes, latency-sensitive UI skill.
Neighbor proposals	**Cheap, cached by graph-state signature**	Useful breadth, not trusted directly.
Rule discovery, eval adjudication, hard flows, onboarding triage	**Frontier — Claude Opus-class**	Low volume, high reasoning value; amortized across customers and releases.

5.1 · The math, worked through

Per feature-crawl, steady state: the executor runs ~25 steps at roughly 3k in / 0.3k out tokens. At mini-tier pricing ($0.15 / $0.60 per 1M) that is about `3000×0.15/1e6 + 300×0.60/1e6 = $0.00063` a step, ×25 ≈ **$0.016**. The few neighbor calls are cached and round to ~$0.001. Everything else — observer, planner, validator, inference, writes — is deterministic and costs nothing. So roughly **$0.017 per feature-crawl**.

Scale that up. A customer with 50 features crawled nightly is `50 × 30 × $0.017 ≈ $26 / customer / month`. At 100 customers, about **$2,600 / month** in inference — and it is dominated by the executor, which is exactly why that call stays on the cheap tier and why validation has to be deterministic. The frontier model is amortized, not per-customer: rule discovery and onboarding triage might be ~100 calls a week across the whole platform, on the order of **$120 / month total** — a rounding error per customer.

Cost lever	What it buys
Cache neighbor calls by graph-state signature	No repeat LLM suggestions when the page and concepts have not changed.
Invalidate instead of full recrawl	You only spend on the bounded subgraph a PR touched.
Small models for the executor	The high-volume call stays cheap.
Reserve the frontier model for offline work	The expensive reasoning is amortized across customers and releases.
Token & latency budgets per stage	Stalls and runaway loops surface as observable failures, not silent spend.

The anti-pattern I refuse is "LLM-judge every step in real time". It makes cost scale with token usage and makes the reasoning non-repeatable — the two things I most want to avoid.

5.2 · Production tooling choices

Tooling should fall out of the boundaries above, not the other way round. I would reach for each of these because of a specific production failure mode it prevents — not because it is fashionable.

Layer	Choice I would start with	Why	What I would avoid
Browser execution	Playwright + constrained LLM executor	Playwright gives deterministic browser control; the LLM only chooses fuzzy targets.	A fully autonomous browser agent with no action contract.
Graph store	Neo4j	Core queries are traversals: CodeArtifact → Selector → Concept → Scenario.	Storing graph-shaped dependencies only in relational tables.

Layer	Choice I would start with	Why	What I would avoid
Artifact storage	S3 / GCS / blob storage	DOM, screenshots and traces are large and tenant-sensitive; Neo4j stores refs only.	Putting large DOM/screenshot blobs inside Neo4j.
Workflow orchestration	Temporal / durable queue	Crawls are long-running, retryable, stateful workflows.	Fire-and-forget cron jobs.
Search / vector memory	OpenSearch / pgvector / LanceDB (auxiliary)	Useful for selector repair and semantic matching; the graph stays source of truth.	Letting vector similarity replace graph truth.
Observability	OpenTelemetry + run traces + artifact refs	Every decision should be replayable across agent, validator and graph writes.	Plain logs with no trace IDs.
Eval store	Versioned golden apps + labeled trajectories	Needed for prompt/model regression and confidence calibration.	Shipping prompt changes without canaries.

Tools are replaceable; the important design constraint is that the graph, evidence store, workflow engine and evaluator have separate ownership boundaries.

6 · A hundred customers, kept apart

My default here is to over-isolate, and I'm comfortable saying so. In a tool that crawls people's logged-in accounts, a forgotten "which tenant is this?" filter isn't a tidy little bug — it's a customer-data incident with someone's name on it. So every query is scoped, every customer's data lives apart, and the only thing allowed to cross between customers is anonymized, general pattern — never one customer's actual screens or scenarios.

Layer	Isolated per tenant	Shared
Graph	Tenant/app/feature partition; database-per-tenant for large accounts	Schema + inference engine (code, not data)
Artifacts	Per-tenant object-store namespace, retention, redaction	Storage implementation
Models/prompts	Versions visible in traces; customer vocabulary stays isolated	Generic executor prompt templates
Learning	No selectors, DOM, screenshots or scenarios cross tenants	Aggregated, reviewed structural priors only
Ops	Per-tenant budgets and throttles	Global monitoring + anomaly detection

Cross-customer learning is deliberately fenced in. We can learn structural priors — "checkout usually has an address step before payment", selector-stability heuristics, candidate inference rules — but a customer's concepts, selectors, scenarios and DOM never cross the boundary. The only thing that moves is anonymized, aggregated pattern, and only after review. Put bluntly: one retailer's cart is not evidence about another retailer's checkout. Cross-customer learning sharpens heuristics; it does not copy concepts.

7 · Operations and observability

Agentic systems fail in ways ordinary services do not: same prompt, different outcome; a correct click that lands on the wrong state; false confidence; a slow model; a provider-side regression you did not cause. So observability has to capture reasoning, artifacts, cost and decisions together, not just latency and error codes.

Telemetry	Example fields	Why it matters
Trace id per run/intent	tenant, app, feature, run_id, step, model_version, prompt_version	Reconstruct a failure across every agent that touched it.
Decision audit	frontier score, graph priority, safety decision, confidence	Explain why an action ran or was blocked.
Artifact refs	DOM snapshot, screenshot, browser-use trace, transition	Human debugging and eval labeling.
Cost / latency	tokens, model, cache hit, duration, retries	Budget governance and customer pricing.
Health metrics	wander rate, validation-failure rate, stale-concept count, flake rate	Catch degradation before customers do.

The alerts I would actually wire up: executor wander rate climbing after a model upgrade; validation failures spiking on one feature right after a PR; scenario confidence decaying past the customer-facing threshold without enough retest capacity to catch up; a cross-tenant query guard rejecting a query that forgot its tenant scope; and cost-per-crawl blowing past budget because of a retry loop. The prototype already emits a good chunk of this — wander detection, validation overrides, per-source accounting — in its run report.

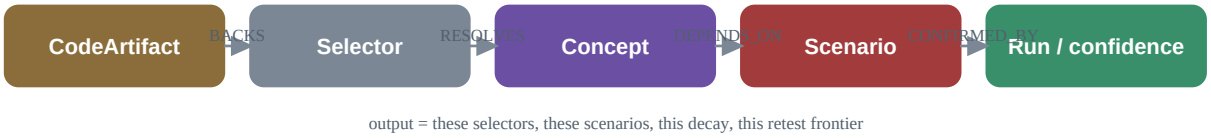
7.1 · Test-account and environment strategy

A production agentic testing platform cannot depend on arbitrary customer production accounts. Each tenant needs dedicated test accounts with seeded data, synthetic payment methods, reset hooks and cleanup policies, so a crawl starts from a known state and leaves no residue behind. For commerce flows the safest target is a staging or sandbox environment with production-like UI but non-production payment rails. And if a customer can only grant production access, risky actions are downgraded to observe-only unless there is an explicit sandbox guarantee — the platform never gets to assume an environment is safe to mutate.

8 · PR blast radius

The optional PR hook is not a separate feature; it falls straight out of the same graph. A PR is an invalidation event. When it touches a component, route, selector or symbol, it invalidates a bounded subgraph, and the blast-radius answer is a traversal.

Figure 5 — PR blast radius is a traversal, not a "rerun everything".



Step	Question it answers
CodeArtifact changed	What paths/symbols did the PR touch?
CodeArtifact → Selector	Which UI locators/components are backed by this code?
Selector → Concept	Which controls or capabilities are affected?
Concept → Scenario	Which discovered and inferred scenarios are at risk?
Scenario → Run / confidence	Which of those risks are stale, high-confidence, or just validated?

The discipline is in the output. It is never "rerun all the checkout tests". It is: these selectors, these concepts, these scenarios, this much confidence decay, and this retest frontier. The prototype's `pr_blast_radius.py` already

produces the contract-only version of exactly that.

9 · The things I'd refuse to ship

This is the section I'd most expect to get grilled on in a review, and honestly that's why it's here. Saying what you'd block is how you set a quality bar instead of just listing features. None of these are "nice to haves" — they're lines I wouldn't cross, each with the thing I'd need to see before I changed my mind.

Refusal	Why, and what I would need first
A path that can click final purchase/payment — even behind a flag	Flags get flipped. Production-money pages must be structurally incapable of a final submit: an allow-list of actions enforced by the safety supervisor as a separate process. First I would want a red-team that tries to make it buy something and fails every time.
LLM self-report as validation	Every mutating or state-changing action needs a deterministic post-condition. First I would want a validator that cannot be satisfied by model prose — which the prototype already does for cart and checkout.
Nightly full-graph recomputation as the freshness strategy	It is expensive and it hides drift. Invalidate bounded subgraphs instead. First I would want the invalidation path proven to catch a real regression on a golden tenant.
Unreviewed cross-tenant learning	Aggregated priors may cross tenants; concepts, selectors, DOM, screenshots and scenarios may not. First I would want an audit log, a human promotion step, and a privacy review.
Uncalibrated confidence in the UI	Confidence without calibration is just false authority. First I would want the calibration check from §4 passing.
Site-specific hacks in the platform core	App quirks — the prototype's smart-wagon readiness, subtotal parsing, marketplace URL handling — belong in adapters. The core owns state, safety, validation, graph and eval. First I would want the adapter boundary plus two genuinely different apps running through it.

10 · Delivery roadmap

I would ship this in phases that keep the system runnable after every step. The prototype already taught me why: big-bang refactors make agent systems almost impossible to debug.

Phase	Deliverable	Exit criteria
0 · Working slice	One feature, browser-use crawler, graph ingest, an inferred missed scenario	Add-to-cart + quantity validated; inference shown (done in Part A).
1 · Harness hardening	Execution memory, constrained executor, readiness gates, deterministic validator	No duplicate execution; no LLM self-report validation (done).
2 · Living graph	Feature-scoped scenarios, confidence decay, bounded invalidation	Stale selectors/concepts re-enter the retest frontier on their own.
3 · Eval harness	Golden apps, canaries, calibration dashboards	Model/prompt regressions caught before customer rollout.
4 · Multi-tenant beta	Tenant isolation, budgets, artifact retention, observability	100-customer architecture exercised under load and cost tests.
5 · PR blast radius	CodeArtifact → Selector → Concept → Scenario traversal	A PR produces a bounded risk report and a retest plan.

10.1 · Rollout modes

Autonomy is earned, not assumed, so the platform moves through rollout modes rather than switching full execution on from day one:

Mode	What the platform is allowed to do
1 · Observe-only	Crawl and build the graph; no mutating clicks.
2 · Recommend	Surface inferred scenarios and blast radius, but do not execute them.
3 · Gated execution	Safe actions auto-run; mutating/destructive actions require policy approval.
4 · Autonomous safe execution	Only after confidence, flake rate and safety checks pass thresholds.

This lets the platform earn autonomy instead of assuming it.

11 · How Part A grounds every claim here

None of this is hypothetical. Each production claim already has a working seed in the prototype — which is the main reason I am comfortable defending it.

Production claim	Prototype function
Deterministic planner / LLM executor split	GraphGuidedExplorer / BrowserUseIntentExecutor
Executor output is evidence, not truth (re-observe + assert)	_verify_cart_mutation, _checkout_reached_ok
Safety an LLM cannot downgrade	normalize_suggestion (ignores LLM-supplied risk)
Absence as a graph query	seed_expected_concepts, missing_expected_concepts
Deterministic Cypher inference	GraphStore.infer_missed_scenarios + INFERENCE_RULES
Graph reasoning feeds back into exploration	_graph_driven_intents (an inferred scenario was executed)
Per-app adapter need (state / marketplace quirks)	_cart_url_for, _ensure_ready_for_intent
PR blast radius as a graph traversal	pr_blast_radius.py, GraphStore.blast_radius
Quantified graph value	the report's "Graph impact" block

12 · Where I'd plant the flag

If I had to put the whole argument in one line: the valuable thing here was never the clever crawler. It's the memory that builds up around it — what we saw, what we actually proved, what should have been there and wasn't, what changed, what went stale, what a code change put at risk, and why the system believes its own answers. That's why I anchored everything on the living graph and a boring, dependable harness instead of on any particular agent framework.

Get that right and the flashy parts — browser-use, Playwright, whichever model is in fashion, the prompts — all become swappable. The part that's genuinely hard to copy, the part I'd actually be building a moat around, is the graph of proven behaviour and the evaluation machinery that keeps it honest as everything underneath it keeps moving.